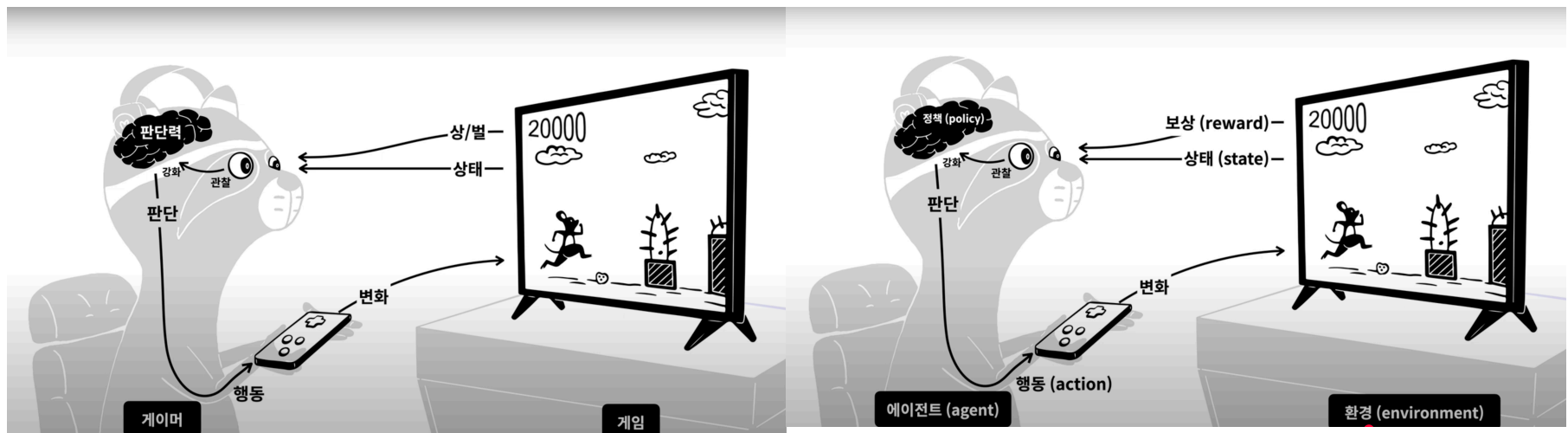


35. RL

35.1 Introduction

RL의 개념

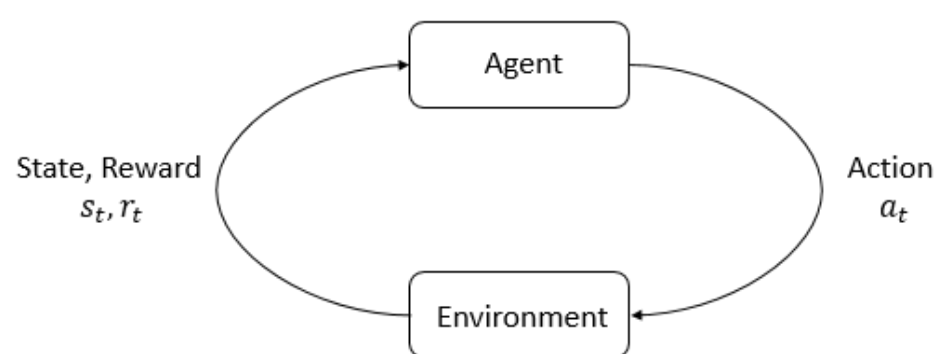
▼ 쉬운 설명



“경험으로 학습한다”

⇒ 상태에 따라서 더 많은 보상을 받을 수 있는 행동을 agent가 할 수 있도록 policy을 만들도록 하는 학습

- 라벨 필요 없음
- 강화학습의 구성 요소
 - 환경(Environment)
 - 에이전트(Agent)
 - 행동(Action)
 - 보상(Reward)
- 강화학습의 작동원리



- agent가 현재 상태 s_t 를 관찰
 - policy에 따라 action a_t 선택
 - 환경은 그에 따른 reward r_t 와 다음 상태 s_{t+1} 반환
-
- 책) 에이전트가 처음에는 알 수 없는 환경과 순차적으로 상호작용(보통 궤적으로 나타남)하면서 학습하는 패러다임
 - (예) 길이 T 의 궤적

$$\omega = (s_0, a_0, r_0, s_1, a_1, r_1, s_2, \dots, s_T)$$

- states s_t , actions a_t , and rewards r_t
- **에이전트의 목표** : action selection policy를 최적화하여, discounted cumulative reward G_0 를 최적화 하는 것

▼ discounted cumulative reward G_0 란?

- 강화학습에서 에이전트는 매 시점마다 **reward** r_t 를 받음
- 그런데 미래의 보상은 현재 시점에서 볼 때 불확실하고, **시간이 지남에 따라 그 가치가 줄어든다**

고 가정

- 단순히 모든 보상의 합을 고려하지 않고, **discount factor** $\gamma \in [0, 1]$ 을 곱해 "현재 가치"로 환산한 보상의 합 사용

→ " discounted cumulative reward "

$$G_0 \triangleq \sum_{t=0}^{T-1} \gamma^t r_t,$$

- $\gamma = 1$: 미래 보상을 전혀 할인하지 않음 → **단순 누적 보상**
- $\gamma = 0$: 현재 보상만 고려 → **즉시 보상만 중시**
- $0 < \gamma < 1$: 시간이 지날수록 미래 보상이 점점 더 작게 반영됨

즉, γ 가 클수록 **장기적 이득을 중시**, 작을수록 **단기적 이득을 중시**

- 길이 T , 할인율 γ
- G_0 은 random variable, 이 챕터에서 **기대값을 최대화**하는 접근을 다룸
- $Q_\pi(s, a) = E[G_t | s_t = s, a_t = a]$

<강화학습에서 사용되는 notation>

1. Value function

- optimal state-value function
 - $V_*(s) = \max_a [R(s, a) + \gamma E_{p(s'|s,a)} [V_*(s')]]$
 - 상태 s에서 시작해서 **최적 정책** 을 따른다고 했을 때, 얻을 수 있는 최대 기대 보상
- optimal action-value function
 - $Q_*(s, a) = R(s, a) + \gamma E_{p(s'|s,a)} [\max_{a'} Q_*(s', a')]$
 - 상태 s에서 특정 행동 a를 했을 때, 이후 최적 정책을 따르면 얻을 수 있는 최대 기대 보상
- V와 Q의 관계
 - $V_*(s) = \max_a Q_*(s, a)$
 - 그 상태에서 가능한 행동들의 가치(Q값) 중 최댓값
- 가치 함수는 Bellman 최적 방정식 형태로 표현

2. Policy

- $\pi_*(s) = \operatorname{argmax}_a Q_*(s, a)$
- 상태 s에서 Q값이 가장 큰 행동 a를 택하는 것

3. Reward

- G_t : 누적보상
- 이 기대값을 최대로 만드는 정책 찾는 것이 강화학습의 목표

RL 방법의 분류

- 강화학습 방법은 보통 다음 기준에 따라 분류된다.
 1. 에이전트가 학습·표현하는 대상
 - 가치 함수(Value Function) / 정책(Policy) / 모델(Model)
 2. 행동 선택 방식
 - **On-policy** : 에이전트가 현재 정책으로 직접 선택한 행동만 학습
 - **Off-policy** : 다른 정책이나 경험으로부터 학습 가능

Method	Functions learned	On/Off	Section
SARSA	$Q(s, a)$	On	Section 35.2.4
Q -learning	$Q(s, a)$	Off	Section 35.2.5
REINFORCE	$\pi(a s)$	On	Section 35.3.2
A2C	$\pi(a s), V(s)$	On	Section 35.3.3.1
TRPO/PPO	$\pi(a s), A(s, a)$	On	Section 35.3.4
DDPG	$a = \pi(s), Q(s, a)$	Off	Section 35.3.5
Soft actor-critic	$\pi(a s), Q(s, a)$	Off	Section 35.6.1
Model-based RL	$p(s' s, a)$	Off	Section 35.4

Table 35.1: Summary of some popular methods for RL. On/off refers to on-policy vs off-policy methods.

- 강화학습은 주로 Markov Decision Process(MDP)를 기반으로 한다
 - MDP에서는 궤적 τ 의 생성 과정을 단일 단계 모델(single-step model)로 분해할 수 있음.
 - planning : 모델 파라미터를 알고 있을 때 → 최적 정책을 구하는 과정
 - learning : 모델을 모를 때 → RL 알고리즘을 사용해 궤적에서 최적 정책을 학습하는 과정
- 모델 사용 유무에 따른 분류
 - model-free RL
 - 모델을 명시적으로 표현하거나 학습하지 않고, 궤적으로부터 직접 정책을 학습
 - model-based RL
 - (first step) 먼저 궤적으로부터 MDP 모델을 학습한 뒤, 정책을 풀기 위한 모델에 planning algorithm 적용
 - (second step) 학습된 모델에서 생성된 가상 경험으로 RL 알고리즘 실행 or planning
- 무엇을 직접 학습하는가에 따른 분류
 - Value-based:** 가치 함수 Q/V함수를 업데이트 → 정책은 Q에서 뽑음.
 - learn the optimal Q-function from experience → derive a policy from it

$$\pi(s) = \arg \max_a Q(s, a)$$

- Policy-based:** 가치 함수 없이 정책 파라미터 직접 업데이트.

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\pi_{\theta})$$

→ 결국 좋은 policy를 만들어내는 것

RL의 핵심난제 Exploration–Exploitation tradeoff

- 에이전트는 항상 두 가지 방식 중 하나로 행동을 선택해야 함
 - Exploitation(이용):** 지금까지 경험으로 **보상이 높다고 아는 행동** 선택 → 당장 이득 큼
 - Exploration(탐험):** 아직 보상이 확실치 않은 행동도 시도 → 더 나은 장기 보상을 발견할 가능성

→ 이 균형을 맞추는 게 **exploration–exploitation tradeoff**

- 35.1.5.1 ϵ -greedy 방법

$$a_t = \begin{cases} \arg \max_a \hat{R}_t(s_t, a), & \text{probability : } 1 - \epsilon \\ \text{random action,} & \text{probability : } \epsilon \end{cases}$$

- notation
 - action a , state s , reward R , time t , 궤적 τ
- 가장 단순하고 흔한 탐험 기법
- 확률 ϵ 만큼은 랜덤 행동을 하도록 함 → 에이전트가 모든 상태–행동 조합을 지속적으로 탐험하도록 보장
- 다만, 랜덤 탐험할 때 좋은 행동과 나쁜 행동을 구분하지 않고 모두 균등 확률로 선택 → 비효율적

- 35.1.5.2 Boltzmann exploration

$$\pi(a | s) = \frac{\exp(\hat{R}_t(s, a)/\tau)}{\sum_{a'} \exp(\hat{R}_t(s, a')/\tau)}$$

- notation
 - action a , state s , reward R , time t , 꺾적 τ
- 확률 분포를 softmax(볼츠만 분포)로 설정
 - ▼ $\tau > 0$ 은 temperature 파라미터
 - $\tau \rightarrow 0$: greedy 정책(거의 최적 행동만 선택)
 - $\tau \rightarrow \infty$: 거의 균등 정책 (모든 행동 비슷하게 탐험)
- 더 유망한 행동(예상 보상이 높은 행동)에 더 높은 확률을 주고, 덜 유망한 행동에는 낮은 확률을 줌.

• ϵ -greedy 방법 vs Boltzmann exploration

예상 reward 값		ϵ -greedy		Boltzmann exploration	
$\hat{R}(s, a_1)$	$\hat{R}(s, a_2)$	$\pi_\epsilon(a s_1)$	$\pi_\epsilon(a s_2)$	$\pi_\tau(a s_1)$	$\pi_\tau(a s_2)$
1.00	9.00	0.05	0.95	0.00	1.00
4.00	6.00	0.05	0.95	0.12	0.88
4.90	5.10	0.05	0.95	0.45	0.55
5.05	4.95	0.95	0.05	0.53	0.48
7.00	3.00	0.95	0.05	0.98	0.02
8.00	2.00	0.95	0.05	1.00	0.00

Table 35.2: Comparison of ϵ -greedy policy (with $\epsilon = 0.1$) and Boltzmann policy (with $\tau = 1$) for a simple MDP with 6 states and 2 actions. Adapted from Table 4.1 of [GK19].

- ϵ -greedy : 가장 큰 Reward를 주는 action = 95% 확률, 나머지 행동들 = 5% 확률로 균등 분배
- Boltzmann exploration : softmax 계산

• 35.1.5.3 Upper Confidence Bounds (UCB) & Thompson Sampling

(a) UCB

- 현재 state에서 모든 action의 Q-value에 대한 upper confidence bounds(상한 신뢰 구간)를 추정 → 가장 높은 UCB 점수를 가진 action 선택
 - ▼ Q-value : 어떤 상태에서 특정 행동을 했을 때, 앞으로 얻을 수 있는 누적 보상의 기대값

$$Q(\text{state}, \text{action}) = \text{reward}$$
- Q-value의 UCB를 얻는 방법
 - use count-based exploration

$$\hat{r} = r + \frac{\rho}{\sqrt{N_{s,a}}}$$

- Notation
 - $N_{s,a}$: state s 에서 행해진 action a 의 수
 - $\rho \geq 0$: 가중치 값
 - 자주 안해본 행동(=N 작으면)이라면 exploration bonus 값이 커져서 더 선택하게 만든다

(b) Thompson Sampling

- 보상 및 전이 모델 파라미터의 posterior 분포를 유지하는 방식으로 유사하게 적용될 수 있다고 간단하게 소개

35.2 Value-based RL

- 가치 함수(value function, V 또는 Q)**를 학습하고, 그 가치 함수에서 **정책을 유도**하는 방법
→ 그 가치 함수에서 정책을 유도함

35.2.1 Monte Carlo RL

*monte carlo sampling : 반복된 무작위 추출을 이용하여 함수의 값을 근사

- Q 함수를 추정하는 가장 간단한 방법
: action $a \rightarrow$ 나머지 궤적을 정책 π 에 따라 샘플링 $\rightarrow$ discounted reward 합 평균
Monte Carlo estimation of the value function

- 궤적 종료 시점
 - terminal state에 도달했을 때
 - 할인율 γ^t 가 무시할 만큼 작아지는 경우
- 이 방법은 policy iteration과 함께 쓰임
 - new and improved policy : 다음 정책을 Q 함수 기준으로 업데이트

$$\pi_{k+1} = \operatorname{argmax}_a Q_k(s, a)$$

where Q_k is approximated using MC estimation

- 모든 (s, a) 조합에 대해 데이터를 모아야 Q 가 수렴
→ 무조건 optimal action만 고르면 안됨
→ ϵ -greedy 사용
- 만약 value function for the optimal policy를 찾고 싶다면 importance sampling을 이용하면 됨
- 단점
 - 많은 궤적을 unroll해야 하기 때문에 $Q(s, a)$ 의 추정치가 분산이 매우 큼
 - 종료 시점을 정할 수 있는 episodic tasks (or finite horizon truncation of continuing tasks)로 제한됨

35.2.2 Temporal difference(TD) learning

- MC RL보다 더 효율적인 방법
 - 기본 아이디어
 - 궤적 전체가 아니라, 샘플된 state나 state-actions의 Bellman 오차를 점차 줄여나가는 방식
- ▼ Bellman 오차
- 이론적으로 가치 함수는 **Bellman 방정식**을 만족해야 함. 여기서 발생하는 오차

$$V(s) = E[r + \gamma V(s')]$$

- 업데이트 규칙
 - 상태 (s, a, r, s') 이 주어졌을 때($a \sim \pi(s)$), $V(s)$ 을 bootstrapping target 쪽으로 이동시키도록 함

$$V(s_t) \leftarrow V(s_t) + \eta[r_t + \gamma V(s_{t+1}) - V(s_t)]$$

- notation 추가
 - η : learning rate

- action a , state s , time t , 가치함수 V
- TD error : $r_t + \gamma V(s_{t+1}) - V(s_t)$
- 목표 값 $r_t + \gamma V(s_{t+1})$ 이 가치 함수 추정치로 정의 → Bootstrapping

35.2.3 TD learning with eligibility traces

- MC와 TD의 가장 큰 차이점은 return을 추정하는 방법
given a trajectory $\tau = (s_0, a_0, r_0, s_1, \dots, s_T)$
 - MC : end of episode나 T가 매우 커질 때까지 기다렸다, $G_{t:T} = r_t + \gamma r_{t+1} + \dots$ 으로 추정
 - TD : one-step lookahead($G_{t:t+1} = r_t + \gamma V(s_{t+1})$)로 추정
- 이 두가지를 interpolate하는 것도 가능
 - (1) n-step rollout 수행

$$G_{t:t+n} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$$

- (2) TD update

$$V(s_t) \leftarrow V(s_t) + \eta [G_{t:t+n} - V(s_t)]$$

- λ - return
 - 특정한 lookahead 값 n 을 고르는 대신, 모든 가능한 값을 가중 평균할 수도 있음

$$G_t^\lambda \triangleq (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

- $\lambda \in [0,1]$
- $(1 - \lambda) = (1 + \lambda + \lambda^2 + \dots)^{-1}$: convex combination of n-step return 보장

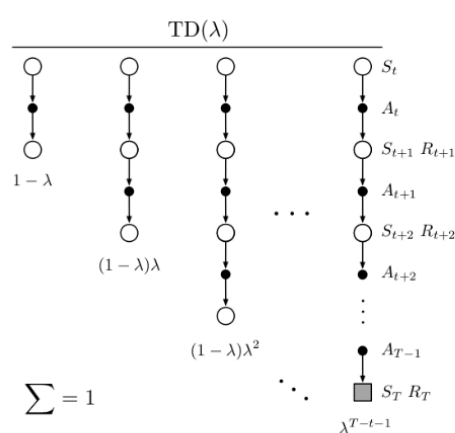


Figure 35.3: The backup diagram for $TD(\lambda)$. Standard TD learning corresponds to $\lambda = 0$, and standard MC learning corresponds to $\lambda = 1$. From Figure 12.1 of [SB18]. Used with kind permission of Richard Sutton.

- important benefit : eligibility traces를 이용하면 efficient하게 계산할 수 있음
→ $TD(\lambda)$ 라고 부른다

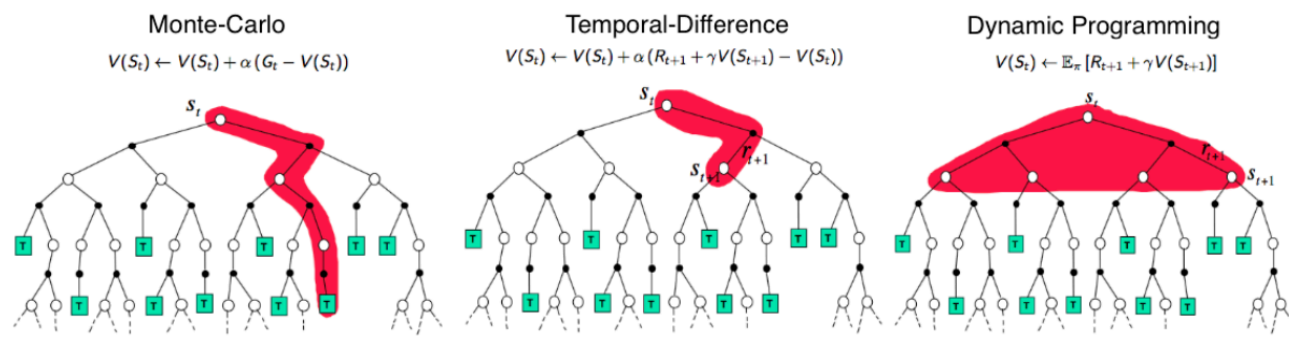


Figure 35.2: Backup diagrams of $V(s_t)$ for Monte Carlo, temporal difference, and dynamic programming updates of the state-value function. Used with kind permission of Andy Barto.

▼ Dynamic Programming

- MDP(Markov Decision Process)의 모델을 안다고 가정하는 model based RL
- 한 step만 보지만, 모델을 알고 있으므로 모든 경우의 수(모든 가능한 다음 상태)를 고려해서 업데이트

35.2.4 SARSA: on-policy TD control

- TD learning은 fixed policy에 대한 value function 추정하는 것
→ policy evaluation을 위한 기법
- optimal policy를 찾는 데에, TD를 활용할 수 있음
 - agent는 매 스텝마다 action을 고를 때 policy π 를 따름
 - TD 업데이트 규칙

$$Q(s, a) \leftarrow Q(s, a) + \eta [r + \lambda Q(s', a') - Q(s, a)]$$

$$\blacksquare a' \sim \pi(s')$$

- Q가 업데이트되면, π 는 Q에 대해 greedy하게 정의되어 있으므로 같이 update
- ϵ -greedy policy 에서 ϵ 를 점차 0으로 줄어가는 것처럼 “greedy in the limit with infinite exploration” (GLIE) policy를 적용하면, optimal Q_* 와 π_* 로 수렴이 보장됨

35.6 Control as Inference

- policy optimization에 대한 다른 접근
→ 확률적 추론으로 접근

35.6.1 Maximum Entropy Reinforcement Learning

- trade-off between reward and entropy maximization
- 새로운 변수 o_t 도입

$$p(o_t = 1 | s_t, a_t) = \exp(\lambda^{-1} R(s_t, a_t)) \quad (35.61)$$

- t의 행동이 최적인지 아닌지를 나타내는 이진 변수
- RL 문제를 **확률적 추론 문제**로 바꿔서 풀겠다는 아이디어
→ “최적의 정책을 찾는 것” ↔ “각 시점에서 $o_t = 1$ 일 확률이 최대가 되는 행동 분포를 추론

→ (일반 RL) 보상 합을 최대화 ↔ (Control as Inference) **trajectory가 모든 $o_t = 1$ 일 확률을 최대화**하는 걸 목표로 함

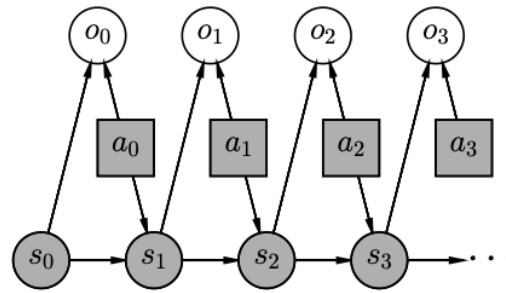


Figure 35.11: A **graphical** model for optimal control. States and actions are observed, while optimality variables are not. Adapted from Figure 1b of [Lev18].

- assumptions
 - $R(s, a) < 0$: (35.61)이 valid probability가 되도록 하기 위해
 - $p(a_t|s_t)$ is uniform action prior : (35.61)에 $p(a_t|s_t)$ 를 넣을 수 있도록
- likelihood of length T trajectory τ

$$\begin{aligned}
 & p(\omega | o_{0:T-1} = 1) \\
 & \propto p(\omega, o_{0:T-1} = 1) \\
 & = p(s_0) \prod_{t=0}^{T-1} p(o_t = 1 | s_t, a_t) p(s_{t+1} | s_t, a_t) \\
 & = p(s_0) \prod_{t=0}^{T-1} p(s_{t+1} | s_t, a_t) \exp\left(\frac{1}{\phi} \sum_{t=0}^{T-1} R(s_t, a_t)\right)
 \end{aligned}$$

- notation
- 전개
 - $p(o_{0:T-1} = 1)$ 은 정규화 상수
 - 확률의 chain rule 이용
 - o_t 대입
- when the state transitions are deterministic
 - $p(s_{t+1}|s_t, a_t) = 1(\tau \text{ is feasible})$

$$p(\tau | o_{0:T-1} = 1) \propto \exp\left(\frac{1}{\lambda} \sum_{t=0}^{T-1} R(s_t, a_t)\right)$$

- $p(s_{t+1}|s_t, a_t) = 0(\tau \text{ is not feasible})$

$$p(\tau | o_{0:T-1} = 0) = 0$$

⇒ minimizing reward = maximum $p(\tau | o_{0:T-1} = 1)$

- optimal policy in this probabilistic model

$$\begin{aligned}
p(a_t | s_t, o_{t:T-1} = 1) &= \frac{p(s_t, a_t | o_{t:T-1} = 1)}{p(s_t | o_{t:T-1} = 1)} \\
&= \frac{p(o_{t:T-1} = 1 | s_t, a_t) p(a_t | s_t) p(s_t)}{p(o_{t:T-1} = 1 | s_t) p(s_t)} \\
&\propto \frac{p(o_{t:T-1} = 1 | s_t, a_t)}{p(o_{t:T-1} = 1 | s_t)}
\end{aligned}$$

▼ 분자 전개(분모도 마찬가지로)

$$\begin{aligned}
&= \frac{p(s_t, a_t | o_{t:T-1} = 1)}{p(o_{t:T-1} = 1 | s_t, a_t) p(s_t, a_t)} \\
&= \frac{p(o_{t:T-1} = 1)}{p(o_{t:T-1} = 1 | s_t, a_t) p(a_t | s_t) p(s_t)} \\
&= \frac{p(o_{t:T-1} = 1)}{p(o_{t:T-1} = 1 | s_t)}
\end{aligned}$$

- 분자 = (현재 보상 기반 likelihood) × (전이 확률) × (다음 state에서의 미래 최적 확률)
- 분모 : "가능한 모든 행동에 대해, 그 행동을 취했을 때 미래가 최적일 확률의 평균"

$$\begin{aligned}
p(o_{t:T-1} = 1 | s_t, a_t) &= \int_{\mathcal{S}} p(o_{t+1:T-1} = 1 | s_{t+1}) p(s_{t+1} | s_t, a_t) \exp(\lambda^{-1} R(s_t, a_t)) ds_{t+1} \\
p(o_{t:T-1} = 1 | s_t) &= \int_{\mathcal{A}} p(o_{t:T-1} = 1 | s_t, a_t) p(a_t | s_t) da_t
\end{aligned}$$

- as calculation is expensive, approximate the optimal policy using a parametric form,

$$\pi_{\theta}(a_t, s_t) \approx p(a_t | s_t, o_{t:T-1} = 1)$$

→ **resulted probability of trajectory τ**

$$p_{\theta}(\tau) = p(s_1) \prod_{t=0}^{T-1} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

- optimize θ (정책을 정의하는 parameter) : 근사하는 policy가 실제 optimal policy 분포와 비슷하도록 KL divergence 최소화

$$\begin{aligned}
&D_{KL}(p_{\theta}(\tau) || p(\tau | o_{0:T-1} = 1)) \\
&= -E_{p_{\theta}} \left[\sum_{t=0}^{T-1} \lambda^{-1} R(s_t, a_t) + H(\pi_{\theta}(s_t)) \right] + const
\end{aligned}$$

- notation

- $\lambda^{-1} R(s_t, a_t)$: 원래 RL에서 maximize 하려던 누적 보상

▼ $\mathcal{H}(\pi_{\theta}(s_t))$: 정책의 엔트로피

- 일반적인 엔트로피 정의
 - 임의의 확률분포 $p(x)$ 에 대해, $\mathcal{H}[p] = -\sum_x p(x) \log p(x)$
 - 행동이 무한 개라면 적분 형태
- 정책의 엔트로피
 - $\mathcal{H}(\pi_{\theta}(s)) = -\sum_a \pi_{\theta}(a | s) \log \pi_{\theta}(a | s)$
- 불확실성 반영(다양성 보존)

- *const*: uniform prior $p(a_t|s_t)$ 에 의해 생기는 항으로(θ 와 무관)
 - maximum entropy RL 또는 MERL 라고 부른다.
- 기존 RL은 가장 좋은 action을 고르도록 설계되어 있는데, Control-as-Inference의 목적 함수에는 entropy항이 있으므로 Q 함수 수정 필요
 - softmax version of the Bellman equation으로 수정
 - $Q_*(s_t, a_t)$

$$= \lambda^{-1}R(s_t, a_t) + E_p(s_{t+1} | s_t, a_t)[\log \int_A \exp(Q_*(s_{t+1}, a_{t+1}))da]$$
 - $Q_*(s_T, a) = 0$: 마지막 시점 T 에서는 더 이상 미래 보상 없음
 - $\lambda \rightarrow 0$, softmax policy becomes greedy, (= standard RL setting)

⇒ Q-function에 entropy 관련 항을 추가하는 것이 key point !

35.6.2 Other approaches

VIREL (Variational Inference for RL)

- alternative model to maximum entropy RL
- (Maximum Entropy RL에서는 행동이 최적일 확률 정의)
- VIREL에서는 **Q값의 상대적인 크기 이용**
 - $Q(s_t, a_t)$ 가 해당 상태에서 **가장 큰 Q값**에 얼마나 가까운지를 확률로 매핑

$$p(o_t = 1 | s_t, a_t) = \exp\left(\frac{Q_w(s_t, a_t) - \max_a Q_w(s_t, a)}{\lambda_w}\right)$$

- 차이가 0일 때 지수 값이 가장 큼
- notation
 - Q_w : approximate action-value function
 - π_w : a stochastic policy
 - o_t : a binary optimality random variable at time t
 - λ_w : temperature parameter, part of parameterization(값이 크면 탐색 증가, 반대는 greedy)
 - critic parameter w , actor parameter θ
 - 정책을 직접 학습할 때에 critic과 actor라는 개념이 필요하다.
 - ▼ critic의 개념

파라미터 w 를 가진 근사 함수 $Q_w(s, a)$ (또는 $V_w(s)$)를 학습하여 현재 정책 π_θ 의 성능을 평가하는 역할
 - ▼ actor의 개념

파라미터 θ 를 가진 정책 $\pi_\theta(a|s)$ 를 학습하여 기대 목적 함수를 최대화하는 역할

$$J(\theta) = E_{\pi_\theta}[Q_w(s, a) + \lambda H(\pi_\theta(\cdot | s))]$$

- EM 알고리즘으로 학습
 - 최대화하고자하는 objective function

$$L(w, \theta) = E_{p(s)}[E_{\pi_\theta(a|s)} \frac{Q_w(s, a)}{\lambda_w} + H(\pi_\theta(s))]$$

- Maximum Entropy RL과 유사한 구조
- E step(policy improvement)
 - critic parameter w 고정
 - actor parameter θ 업데이트(using gradient ascent with step size η_θ)

$$\theta \leftarrow \theta + \eta_\theta \nabla_\theta L(w, \theta)$$

- M step(policy evaluation)
 - actor parameter θ 고정
 - critic parameter w 업데이트

$$w \leftarrow w + \eta_w \nabla_w L(w, \theta)$$

기타 방법들

- 목표 : policy parameter θ 최적화 통해 궤적 τ 의 기댓값 G 최대화

$$J(\pi_\theta) = \int G(\tau) p(\tau|\theta) d\tau$$

- $G(\tau)$ 를 확률 밀도 함수로 다루고, pseudo-likelihood function으로 해석
 - $J(\pi_\theta)$ 를 likelihood maximization 문제로 볼 수 있다(좋은 궤적이면 likelihood 크다고 해석)
- Active Inference
 - 뇌과학에서 인기 있는 free energy principle에 기반
 - state estimation(perception)과 parameter estimation(learning)을 동시에 추정
 - variational Bayes EM과 같은 구조